

⚙️ DEVOPS & SETUP (Настройка Окружения)

Дата: 2026-06-26 | **Версия:** 2.0

Автор: Koda (AI) + NLP-Core-Team

Назначение: Инструкция по запуску проекта. Архитектура узлов, порты, контейнеризация, тесты.

1. АРХИТЕКТУРА ПРОЕКТА — MONOREPO

Инструмент: **pnpm** (Workspace) + **Turborepo** (опционально).

Архитектура: Моноrepo из нескольких узлов, объединённых кодовой базой.

Каждый узел может подниматься на одном или более контейнерах на одном или более серверах в рамках одной виртуальной сети и одного workspace.

```
balloo-platform/
├── apps/
│   ├── web/           # Next.js (Основной клиент) – Узел Web App
│   ├── mobile/        # React Native (iOS/Android) – Узел Mobile API
│   ├── desktop/       # Electron (Windows/Mac/Linux) – Узел Desktop
│   └── admin/         # Next.js (Панель администратора) – Узел Admin
├── Portal
│   ├── packages/
│   │   ├── ui/        # Shared UI Components (Tailwind)
│   │   ├── core/      # Shared Logic (Utils, Constants)
│   │   ├── db/        # Database Schema & Migrations (Drizzle ORM)
│   │   └── api/       # API Client (Fetch wrappers, Types)
│   ├── services/
│   │   ├── api-gateway/ # API Gateway (Auth, Routing) – Узел API
│   │   ├── messenger/  # WebSocket Messenger (Real-time) – Узел Messenger
│   │   └── media-server/ # Обработка файлов (FFmpeg, Thumbnails) – Узел
│   └── Media
│       ├── sms-node/   # SMS Gateway (Android Nodes) – Узел SMS
│       └── docker/
│           ├── docker-compose.yml # Local Dev (Postgres, Redis)
│           └── Dockerfile         # Production Images
├── .env                # Environment Variables (GitIgnored)
├── .env.example        # Template for variables
├── package.json        # Root scripts
└── pnpm-workspace.yaml # Workspace config
```

2. ПОРТЫ УЗЛОВ (30**)

Узел	Порт	Назначение
API Gateway	3001	Auth, Routing, REST API
Messenger	3002	WebSocket, Real-time сообщения
Admin Portal	3003	Панель администратора
Web App	3004	Next.js клиент
Mobile API	3005	React Native API
Database	3006	PostgreSQL (внутренний)
Cache	3007	Redis (внутренний)
File Storage	3008	Media Server (внутренний)

Все узлы работают в одной виртуальной сети Docker (**balloo_net**) в рамках одного workspace.

3. ENVIRONMENT VARIABLES (**.env**)

Расположение: Корень проекта (для Backend) и **apps/web/.env** (для Frontend).

3.1 Backend (Services)

```
# --- DATABASE ---
DB_HOST=localhost
DB_PORT=5432
DB_NAME=balloo
DB_USER=balloo_user
DB_PASS=secure_password_123
DB_SSL=false # true для production

# --- REDIS (Cache & Queues) ---
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASS=

# --- AUTH & SECURITY ---
JWT_SECRET=super_secret_jwt_key_min_32_chars
JWT_REFRESH_SECRET=super_secret_refresh_key_min_32_chars
BCRYPT_ROUNDS=12

# --- YANDEX DISK (Storage) ---
YANDEX_DISK_TOKEN=your_oauth_token
YANDEX_DISK_FOLDER=/balloo_files

# --- SMS NODES (Android) ---
SMS_NODE_API_KEY=your_node_api_key
```

```
# --- EMERGENCY ALERTS ---
EMERGENCY_BOT_TOKEN=bot_token_for_mchs
EMERGENCY_WHITELIST_IPS=192.168.1.1,10.0.0.1

# --- SERVER ---
PORT=4000
NODE_ENV=development
```

3.2 Frontend (Web - `apps/web/.env`)

```
# --- API ---
NEXT_PUBLIC_API_URL=http://localhost:4000
NEXT_PUBLIC_WS_URL=ws://localhost:4000/socket

# --- APP INFO ---
NEXT_PUBLIC_APP_NAME=Balloo
NEXT_PUBLIC_VERSION=1.0.0

# --- STORAGE ---
NEXT_PUBLIC_MAX_FILE_SIZE=52428800 # 50MB
```

4. DOCKER COMPOSE (Local Development)

Файл: `docker/docker-compose.yml`

```
version: '3.8'

services:
  # PostgreSQL 16
  postgres:
    image: postgres:16-alpine
    container_name: balloo_db
    restart: unless-stopped
    environment:
      POSTGRES_USER: balloo_user
      POSTGRES_PASSWORD: secure_password_123
      POSTGRES_DB: balloo
    ports:
      - "5432:5432"
    volumes:
      - pg_data:/var/lib/postgresql/data
    networks:
      - balloo_net

  # Redis 7
  redis:
```

```
image: redis:7-alpine
container_name: balloo_redis
restart: unless-stopped
ports:
  - "6379:6379"
volumes:
  - redis_data:/data
command: redis-server --appendonly yes
networks:
  - balloo_net

volumes:
  pg_data:
  redis_data:

networks:
  balloo_net:
    driver: bridge
```

5. УСТАНОВКА И ЗАПУСК

5.1 Установка зависимостей

```
# 1. Установка pnpm (если нет)
npm install -g pnpm

# 2. Установка зависимостей во всех пакетах
pnpm install
```

5.2 Запуск Базы Данных и Redis

```
# Запуск Docker контейнеров
docker-compose -f docker/docker-compose.yml up -d

# Проверка статуса
docker-compose -f docker/docker-compose.yml ps
```

5.3 Инициализация Базы Данных — Drizzle ORM

```
# Генерация миграций из schema (Drizzle ORM)
pnpm --filter @balloo/db migrate:generate

# Применение миграций
```

```
pnpm --filter @balloo/db migrate:up
```

```
# Seed данных
```

```
pnpm --filter @balloo/db seed
```

Drizzle ORM — TypeScript-native, light-weight, генерирует миграции из schema. Идеален для Next.js + PostgreSQL.

5.4 Тесты — Vitest + Testing Library

```
# Запуск всех тестов
```

```
pnpm test
```

```
# Запуск с покрытием
```

```
pnpm test:coverage
```

```
# Backend цель: 70%+
```

```
# Frontend цель: 60%+
```

Vitest + Testing Library — быстрый, native ESM, работает с Next.js.

Если ИИ-кодогенератор сможет создать и проверить тесты без участия пользователя — внедрять.

5.5 Запуск Серверов

```
# Backend (Max Server)
```

```
pnpm --filter @balloo/max-server dev
```

```
# Frontend (Web)
```

```
pnpm --filter @balloo/web dev
```

5.6 Запуск всего сразу (Turborepo)

```
pnpm dev
```

6. ПРОДУКЦИОННЫЙ ДЕПЛОЙ (Production)

6.1 Сборка

```
# Backend
pnpm --filter @balloo/api-gateway build

# Frontend
pnpm --filter @balloo/web build
```

6.2 Docker Build

```
docker build -t balloo-api -f services/api-gateway/Dockerfile .
docker build -t balloo-web -f apps/web/Dockerfile .
```

6.3 Требования к серверу

- **CPU:** 2+ ядра (для шифрования/дешифровки E2E).
- **RAM:** 4GB+ (PostgreSQL + Node.js + Redis).
- **Disk:** SSD (для быстрой записи сообщений).
- **OS:** Linux (Ubuntu 22.04 LTS / Debian 12).
- **Сеть:** Все узлы в одной виртуальной сети **balloo_net**.

END OF DEVOPS SETUP